Puzzroo Flagship Engineering Audit Report

Date: July 29, 2026 | **Audit Level:** Enterprise / FAANG / Principal Engineer
Overall Production Readiness Score: 5.8/10 | Overall Grade: C+
Launch Recommendation: CONDITIONAL — Limited Beta Only

Table of Contents

---

1. Executive Summary

The Puzzroo platform is a Next.js 16 polygame application offering Sudoku, CrossMath, Nonogram, and Tangram puzzles, with Chess, user accounts (Firebase + custom JWT), billing (Stripe), and analytics. The platform has a solid foundation in authentication design, input validation (Zod), rate limiting, and security headers. However, significant gaps remain in DevOps infrastructure, testing coverage, production readiness, and scalability architecture.

****Overall Production Readiness Score: 5.8/10 — Grade: C+****

Category Scores

Category | Score | Grade
----------|-------|-------
Architecture | 7/10 | B
Backend | 6.5/10 | B-
Security | 7.5/10 | B+
Authentication | 8/10 | A-
Database | 5/10 | C
API | 7/10 | B
Games | 6/10 | C+
Performance | 4/10 | D
Testing | 3/10 | D
DevOps | 2/10 | D
Observability | 3/10 | D
Maintainability | 6/10 | C+
Scalability | 3/10 | D
Documentation | 5/10 | C
Operational Readiness | 2/10 | D

Launch Recommendation: CONDITIONAL — The platform can launch for limited beta testing but requires critical remediation in DevOps, testing, and performance before general availability.

---

2. Architecture Audit

2.1 Technology Stack

Layer | Technology | Assessment
-------|-----------|------------
Framework | Next.js 16 (App Router) | Modern, SSR-capable
Language | TypeScript 5.x | Strict mode enabled
Database | MongoDB via Mongoose | Appropriate for document model
Auth (Frontend) | Firebase Auth SDK v12 | Solid SDK
Auth (Backend) | Firebase Admin + Custom JWT | Good hybrid approach
Auth Tokens | jsonwebtoken (HS256) | Standard
Password Hashing | bcryptjs (cost 10) | Adequate
API Layer | Next.js Route Handlers | Appropriate
Validation | Zod v4 | Best-in-class
State Management | React Context + TanStack Query | Standard
Styling | Tailwind CSS 3.x + PostCSS | Standard
Email | react-email + nodemailer | Adequate
Payments | Stripe SDK | Industry standard
Testing | Vitest 2.x + Testing Library | Adequate
Deployment | Vercel (implied by Next.js) | Standard

2.2 Architecture Observations

Strengths:
- Clean separation of concerns between client API layer (src/lib/api/) and server-side logic (src/lib/server/)
- Shared puzzle data and libraries in shared/ package prevent duplication across games
- Consistent route helper pattern (withAuth wrapper) across all game API routes
- Zod validation schemas at the API boundary provide strong input contracts
- Server-side puzzle verification means clients cannot spoof completion
- The puzzle registry pattern (getGameRegistry) enables adding new games via configuration
- The withAuth error mapping provides consistent error codes and status codes across all game routes
- Four games follow a consistent session lifecycle (start/save/complete/verify/pause/resume/restart/abandon/replay)

Weaknesses:
- Dual-path architecture in src/lib/server/puzzles/: CrossMath, Nonogram, and Tangram use the newer modular service pattern (services/ subdirectory), while Sudoku uses the older src/lib/server/services/sudoku/ pattern — increasing maintainability burden
- src/data/ directory duplicates puzzle data that exists in shared/src/data/, creating a maintenance hazard
- Multiple PlaySession models exist (CrossMathPlaySession, NonogramPlaySession, SudokuPlaySession, TangramPlaySession) plus a generic top-level PlaySession model that appears unused

- The src/lib/server/games/ directory contains shared infrastructure (completion.ts, migration.ts, subscriptions.ts) that is only partially utilized by the game routes
- No generic game session abstraction — each game reimplements the same session lifecycle independently
- The withAuth wrapper is duplicated in each game's route-helpers.ts file
- Rate limit key naming is inconsistent across games (crossmath-verify, sudoku-complete, nonogram-complete vs games:verify pattern)

2.3 SOLID Principles Assessment

- Single Responsibility: Most service classes follow SRP well (SessionService, VerificationEngine, StatisticsService each have focused responsibilities). The auth route handler violates SRP by combining 11 distinct actions in one ~550-line function.
- Open/Closed: The puzzle registry pattern (getGameRegistry) is open for extension but requires manual registration in registry.ts — no auto-discovery.
- Liskov Substitution: The withAuth wrapper provides a consistent interface but error handling varies across route handlers.
- Interface Segregation: DTOs and schemas are well-segregated via Zod — each endpoint has its own validated schema.
- Dependency Injection: Limited — most services import directly from models rather than using dependency injection, making unit testing harder and coupling tighter.

2.4 DRY Violations & Code Duplication

1. The session lifecycle pattern

(start/save/complete/verify/pause/resume/restart/abandon/replay) is duplicated across all 4 games with only minor differences — a generic game session abstraction would reduce this by ~90% of duplicated code.
2. The withAuth wrapper pattern is duplicated in route-helpers.ts for each game (crossmath, nonogram, tangram all have their own copy).
3. Rate limit key naming is inconsistent (crossmath-verify vs games:verify pattern vs sudoku-complete).
4. Puzzle data exists in both src/data/ and shared/src/data/ — maintenance risk.
5. The statistics service update pattern is identical logic duplicated per game.
6. Auth route handler uses ~550 lines to handle 11 actions that each follow the same pattern (rate limit -> validate -> auth check -> action -> response).
7. CompleteSession schemas and response types are duplicated across game-specific validator and types files.
\n### 4. Database Audit

4.1 MongoDB Collections

Collection | Model | Key Indexes | TTL
----------|-------|-------------|-----
users | User | username(unique), email(unique sparse), publicId(unique sparse), firebaseUid | No
loginSessions | LoginSession | userId, (userId+deviceFingerprint+status), lastSeenAt | 7 days
playSessions | PlaySession | userId+puzzleId, userId+status, userId+status+completedAt | 90 days
crossmathSessions | CrossMathPlaySession | Various per-game per-user patterns | No explicit

nonogramSessions | NonogramPlaySession | Various per-game per-user patterns | No explicit
sudokuSessions | SudokuPlaySession | Various per-game per-user patterns | No explicit
tangramPlaySessions | TangramPlaySession | Various per-game per-user patterns | No explicit
gameProgress | GameProgress | userId+gameId+puzzleId(unique) | No
dailyChallenges | DailyChallenge | (date+userId unique), date, userId | No
puzzleStatistics | PuzzleStatistics | puzzleId(unique), difficulty | No
userStatistics | UserStatistics | userId+gameId(unique) | No
analyticsEvents | AnalyticsEvent | userId+timestamp, event+timestamp, timestamp | 180 days
contactMessages | ContactMessage | userId, status | No
subscriptions | Subscription | userId(unique), stripeCustomerId(sparse) | No
transactions | Transaction | userId, stripePaymentIntentId(sparse) | No
emailPreferences | EmailPreference | userId(unique) | No

4.2 Database Observations

****Strengths:****
- TTL indexes on LoginSession (7 days) and AnalyticsEvent (180 days) for automatic data cleanup
- Compound indexes on frequently queried patterns (userId+gameId, userId+puzzleId)
- Unique indexes on publicId, email, username to prevent duplicates
- Sparse indexes on email for guest accounts without emails
- Index on lastLoginAt for user activity queries

****Weaknesses:****

- No backup strategy documented — no mention of MongoDB backups, snapshots, or replica sets
- No migration framework — schema changes are ad-hoc with post-hooks on User model
- Connection pool limit of 10 — insufficient for moderate concurrency under load
- No read replicas — all queries hit the same primary
- No sharding strategy for horizontal scaling
- Analytics events can grow unbounded between TTL sweeps (up to 180 days)
- No denormalization strategy — user statistics are computed on-the-fly via aggregation rather than cached
- Puzzle solutions stored directly in puzzle documents — if a puzzle is found to be flawed, there is no easy way to update solutions after deployment

4.3 Query Efficiency Risks

- Streak calculation queries ALL completed sessions per user on every completion — O(n) per completion, runs sequentially
- Daily puzzle selection uses $sample aggregation which is efficient for random selection
- Leaderboard pagination uses skip/limit which degrades at scale (skip 10000 scans 10000 rows)
- StatisticsService.recomputeOnEveryCompletion queries the entire completed sessions collection per user

5. Security Audit

5.1 OWASP Top 10 Assessment

Category | Status | Severity | Notes
----------|--------|----------|-------
A01 Broken Access Control | MEDIUM | Medium | Guest role can access some game APIs; no explicit resource-level ownership checks on all session routes
A02 Cryptographic Failures | LOW | Low | bcrypt cost 10 adequate for passwords; JWT uses HS256 with configurable secrets
A03 Injection | LOW | Low | Mongoose ODM parameterized queries; Zod validates all inputs; no direct MongoDB expression injection
A04 Insecure Design | MEDIUM | Medium | No idempotency keys; CSRF protection falls back to spoofable custom headers; no explicit threat modeling
A05 Security Misconfiguration | HIGH | High | No Dockerfile; no CI/CD security scanning; no dependency vulnerability scanning; no CSP nonce; no security headers in dev
A06 Vulnerable Components | MEDIUM | Medium | Dependencies not scanned in this audit; eslint v8 is EOL; requires ongoing scanning
A07 Auth Failures | LOW | Low | Strong auth design with rotation, brute force protection, session revocation
A08 Software Integrity | MEDIUM | Medium | No supply chain scanning; no SLSA provenance
A09 Logging Failures | MEDIUM | Medium | Audit logging exists but is fire-and-forget with silent failure swallowing; no centralized logging
A10 SSRF | LOW | Low | No SSRF vectors observed in server-side code

5.2 JWT Security

- Access tokens: HS256, 15-minute expiry (configurable via ACCESS_TOKEN_EXPIRES)
- Refresh tokens: HS256, 7-day expiry (configurable via

REFRESH_TOKEN_EXPIRES)
- Algorithm explicitly restricted to HS256 (no algorithm confusion possible)
- JWT_SECRET checked for minimum 16 chars but not for entropy/quality
- JWT_SECRET checked for minimum length (16 chars) with a warning if below 32 chars — but execution continues anyway even if weak
- Refresh tokens contain jti (session ID) and ver (token version) for rotation detection
- Reuse detection: if a tokenVersion mismatch occurs, the session is revoked and all tokens for that user become invalid
- Access tokens are NOT httpOnly — they are stored in memory (React state), which is good for XSS resistance but means a refresh is needed on every page load
- Refresh tokens ARE in httpOnly, Secure, SameSite=Strict cookies — good for CSRF resistance
- No revocation list maintained server-side for invalidated tokens (relies on LoginSession status check)

5.3 Authentication & Session Security (Detailed)

****Strengths:****
- Refresh tokens stored in httpOnly, Secure, SameSite=Strict cookies
- Session-based JTI means logout is server-side effective
- Brute force protection with progressive lockout (5 attempts / 5s lock, 10 / 30s lock, 20 / 5min lock)
- Rate limiting on all auth endpoints (3-30 req/min depending on sensitivity)
- OAuth account linking prevents duplicate accounts from different providers
- Guest-to-premium conversion preserves all history (same _id)

- Password change invalidates tokens via token version bump
- "Logout All" revokes all active sessions across all devices
- Email verification flow with 24-hour expiring tokens
- Device fingerprinting (SHA-256 hash of User-Agent header) on sessions
- Geo-location tracking on sessions (geoLocate from IP)
- Auth events logged (login, login_failed, logout, logout_all, etc.)

****Weaknesses:****
- No brute force protection on verification or password reset endpoints (only login and registration protected)
- No account lockout after repeated failed verification attempts
- Email verification tokens are SHA-256 hashed (not bcrypt) — acceptable for short-lived tokens but different from password hashing
- Reset password token expiry not explicitly configured in the auth route handler
- CSRF protection falls back to allowing requests with x-requested-with or x-forwarded-for headers, which can be spoofed by malicious extensions or sites
- No explicit CSRF token cookie set by server-side middleware — the CSRF middleware checks for a mismatch but never sets the cookie itself

5.4 Input Validation (Zod Schemas)

Password policy (registerSchema): min 6 chars, max 20 chars — below NIST/OWASP recommendations (minimum 8+ with complexity requirements for uppercase, lowercase, digits, special characters). No password breach check integrated.

Username validation: 3-20 chars, lowercase alphanumeric + ._- — reasonable.

Email validation: Uses Zod's built-in .email() validator — adequate.

5.5 Content Security Policy (Production)

Production CSP from next.config.js includes 15 directives:
- default-src 'self'
- connect-src 'self' + cloudinary + firebase + stripe + Google APIs
- frame-src 'self' + Google/Facebook login + Stripe
- script-src 'self' 'unsafe-eval' 'unsafe-inline' + Google/Facebook/Stripe scripts
- style-src 'self' 'unsafe-inline'
- img-src 'self' data: blob: + cloudinary + firebase + Google + Facebook CDN
- frame-ancestors 'none' (good — no clickjacking)
- report-uri /api/v1/system/csp-report

****Weakness:** script-src includes 'unsafe-eval' and 'unsafe-inline' which significantly weakens XSS protection. Required for third-party scripts but overly permissive. Recommend nonce-based CSP.**

5.6 File Upload Security

- File uploads go through Cloudinary via multer-storage-cloudinary
- No explicit file type validation visible in upload routes
- No explicit file size limits beyond Next.js body parser defaults
- No virus/malware scanning on uploaded files
- Avatar upload uses Cloudinary which is appropriate for storage but

lacks validation

5.7 Dependency Security

- No Dependabot, Snyk, or security scanning tooling configured
- No npm audit in CI pipeline
- No Software Bill of Materials (SBOM) generation
- jose library is overridden to version 5.9.6 (via package.json overrides) — unusual, may indicate a dependency conflict resolution
- eslint v8 is used — End of Support (ESR) as of 2026, should be on v9
- Firebase Admin v14 is current; Firestore usage not fully inspected but is a known Google-maintained dependency

---

---

6. Authentication Audit

6.1 JWT Lifecycle

Login (email/password) -> bcrypt.compare -> issueSession -> {accessToken, refreshToken}
-> accessToken stored in memory (React state, not localStorage)
-> refreshToken stored in httpOnly Secure SameSite=Strict cookie
-> Access token used for API calls via Authorization: Bearer header
-> On expiry (15 min), client calls POST /api/v1/auth/refresh
-> Refresh token verified against JWT_REFRESH_SECRET, JTI checked against LoginSession
-> Token version incremented (rotation detected by LoginSession.tokenVersion mismatch)

-> Reuse detection: if tokenVersion mismatch occurs, session is revoked
-> New access + refresh tokens issued
-> Old refresh token becomes invalid (single-use)

6.2 Session Management

- Session Creation: IP, UA, device fingerprint, geolocation captured
- Session Revocation: Direct LoginSession.status update
- Session Persistence: 7-day TTL on inactive sessions via expireAfterSeconds
- Session Ownership: JTI in JWT binds token to specific session device
- Replay Protection: Token version increment on refresh; reuse detection
- Device Fingerprinting: SHA-256 hash of User-Agent header
- Concurrent Sessions: New session marks previous isCurrent=false (unless markOthersInactive=false)

6.3 OAuth Flow (Firebase)

Firebase Auth POSTs to /api/v1/auth/[provider] with Firebase ID Token
-> Server verifies with Firebase Admin SDK (verifyIdToken)
-> Finds existing user by firebaseUid OR email match
-> Links OAuth to existing account (email merge preserves publicId)
-> Converts guest accounts in-place (same _id, preserves all history)
-> Brand-new OAuth users get placeholder username + usernameSet=false (routed to /choose-username)
-> Creates LoginSession -> issues JWT tokens bound to session
-> Returns {payload, refreshToken, converted, sessionId}

6.4 Password Reset Flow

The password reset flow uses /api/v1/passwords/[..slug] routes. The authValidator includes forgotPasswordSchema and resetPasswordSchema schemas. The actual implementation of these route handlers was not fully traceable in the file listing — they exist at the /passwords/[..slug] route path but their implementation detail requires further verification.

6.5 Token Leakage Risks

- Access tokens in JS memory (not localStorage) — good for XSS resistance but lose tokens on page refresh
- Refresh tokens in httpOnly cookies — good for CSRF resistance
- 10-minute auth health check interval: fetch /api/v1/auth/refresh every 10 min means up to 10 min of stale token before auto-refresh
- No explicit token invalidation on password change — relies on token version bump via getSessionTokenVersion
- The frontend-auth.ts stores accessToken in a module-level variable (module-scoped, not localStorage), which means it is lost on page reload but also inaccessible to XSS in a strict sense
\n### 7. Performance Audit

7.1 Caching

****Implemented:****
- Cache-Control headers on daily puzzle (86400s / 1 day) and leaderboard (60s / 1 min) endpoints
- TanStack Query client with 5-minute stale time, 10-minute garbage collection, disabled refetch on window focus

Missing:
- No Redis/caching layer for database queries
- No API response caching beyond HTTP headers
- No ETag or conditional request support
- Puzzle data queried from DB on every request — no application-level caching
- No CDN configuration beyond Vercel's edge network (which handles this partially)

7.2 Rate Limiting

Implementation: In-memory sliding window per (key, IP) tuple.

Limitations:
- Does not scale beyond single instance (in-memory, per-process)
- No Redis-backed distributed rate limiting
- Rate limit state lost on server restart
- Granular limits vary: 120 req/min for daily/leaderboard/play, 60 req/min for verify, 30 req/min for complete/resume, 5-10 req/min for sensitive auth actions

7.3 Database Performance

Good:
- Most query patterns have appropriate indexes
- N+1 risks mostly mitigated with $in batched queries (leaderboard resolves usernames in single query)
- $sample aggregation used for daily puzzle selection (efficient)

Concerning:

- Streak calculation queries ALL completed sessions per user on every completion — O(n) per completion
- Connection pool maxPoolSize=10 may be insufficient under concurrent load
- No query profiling enabled
- No slow query logging configured

8. Game Engine Audit

8.1 Sudoku

- Verification: Server-side against stored solution (array comparison of 81 cells) — server-authoritative
- Scoring: Based on time, mistakes, hints used
- Session Lifecycle: Full (start/save/pause/resume/restart/abandon/replay/verify/complete)
- Daily Challenge: Via DailyChallenge model
- History: Separate completed/abandoned history routes
- Statistics: Dedicated statistics service and UserStatistics model
- Architecture Note: Uses older src/lib/server/services/sudoku/ pattern vs newer modular pattern used by other games

8.2 CrossMath

- Verification: Server-side equation-by-equation evaluation (left-to-right evaluation, no PEMDAS) — server-authoritative
- Scoring: correctEquations * 10 * multiplier - mistakes * 5 - hints * 20 (difficulty multiplier applied)
- Session Lifecycle: Complete (start/save/pause/resume/restart/abandon/verify/complete/replay)
- Daily Challenge: Via DailyChallenge model (crossmath-specific

routes exist)
- History: Separate completed/continue/history routes
- Statistics: CrossMath-specific StatisticsService with per-difficulty tracking
- Notes: blinkerPreview field in puzzle responses exposes first 5 blank cells — may reduce puzzle difficulty for users who inspect responses

8.3 Nonogram
- Verification: Server-side row/column clue matching against player grid — server-authoritative
- Scoring: Based on accuracy, time, hints, mistakes via StatisticsService
- Session Lifecycle: Most comprehensive — includes result endpoint and daily progress tracking
- Daily Challenge: Via DailyChallenge model with daily progress tracking
- History: Separate completed/history routes
- Statistics: Nonogram-specific StatisticsService
- Notes: verification validates clue consistency row-by-row and column-by-column

8.4 Tangram
- Verification: Server-side polygon geometry engine with containment, overlap, and boundary checks — server-authoritative
- Scoring: Based on piece correctness, accuracy, elapsed time
- Session Lifecycle: Full set of operations
- Daily Challenge: Via DailyChallenge model

- Statistics: Tangram-specific StatisticsService
- Notes: Tangram geometry engine (src/lib/server/tangram/geometry/engine.ts) is the most sophisticated verification engine — uses polygon intersection detection, containment checks, rotation tolerance, and position snapping with configurable tolerances (TOLERANCE.POSITION, TOLERANCE.ROTATION). This is the strongest verification implementation of any game. Floating-point precision edge cases possible in geometry calculations. No anti-cheat beyond server-side verification (client-side timing can be spoofed).

8.5 Completion Bus (Race Condition Risk)

The completionBus emits events on puzzle completion, and ensureGameSubscriptions called for subscription-based unlocks. The completionBus is an in-memory EventEmitter — events are lost on server restart, no multi-instance support, no dead-letter queue, no persistence. Under concurrent completion requests for the same session, the event bus could emit duplicate events.

9. Dataset Audit

9.1 Dataset Inventory

Game | Difficulties | Pool Size per Difficulty | Source
------|-------------|----------------------------|--------
Sudoku | easy, medium, hard, expert | ~1000 (900+ min) | shared/src/data/sudoku/
CrossMath | easy, medium, hard | ~1000 (900+ min) |

shared/src/data/crossmath/
Nonogram | easy, medium, hard, expert | ~1000 (1000 exact) | shared/src/data/nonogram/
Tangram | easy, medium, hard | ~1000 | shared/src/data/tangram/
Past Puzzles | easy, medium, hard | Small curated set | shared/src/data/pastPuzzles/
CrossMath Patterns | patterns.json | Pattern definitions | shared/src/data/crossmath/patterns.ts

9.2 Integrity Validation

Test files validate structural integrity:
- Sudoku: grid 9x9, solution values 1-9, given cells match solution, unique IDs, givens match solution
- CrossMath: pattern existence, grid dimensions match pattern, blank cells editable+empty, solution covers all NUMBER cells, result cells non-editable
- Nonogram: 1000 puzzles/difficulty, correct sizes per difficulty, unique IDs, clue-solution consistency (generateRowClues/generateColumnClues match), dimensions match declared size
- Tangram: polygon validation via shared tangramValidation.ts

9.3 Weaknesses

- src/data/ duplicates shared/src/data/ — maintenance risk
- No dataset hash verification for production integrity
- No versioning of datasets — puzzle content changes require code redeployment
- Tangram polygon datasets generated by Python scripts but no automated validation in Next.js build pipeline

- No post-hoc difficulty validation (actual solve times vs declared difficulty)
- No dataset migration path for content updates

---

10. TypeScript Audit

10.1 Strict Mode
strict: true enabled in tsconfig.json. noEmit: true (type-only builds). ESLint configured with eslint-config-next. TypeScript 5.x.

10.2 Critical any Usage
- Auth route handler uses let body: any = {} for JSON parsing
- formatUser() takes user: any instead of a typed UserDocument
- authPayload() takes user: any instead of typed UserDocument
- issueSession() takes request: any instead of strongly typed NextRequest
- handleOAuth() takes request: any instead of typed NextRequest
- Multiple service methods use Record for update objects instead of specific types

- Game session save routes use Record for grid state
- Tangram geometry engine returns type any for piece results
- CrossMath/Nonogram/Tangram puzzle document types use unknown casts (as unknown as CrossMathDoc)

10.3 Duplicate Types/Interfaces
- CompleteSessionResponse defined in both server-side (crossmath/types.ts, nonogram/types.ts, tangram/types.ts) and shared locations
- Session/PuzzleResponse types duplicated across game-specific modules
- TrackEventInput type duplicated between trackValidator.ts and trackRoute.ts
- Difficulty type duplicated across games (some use Difficulty, others use string union)

10.4 Weak Typing
- Mongoose hooks use (userSchema as any) casts — expected for Mongoose but should be minimized
- toResponse mappers use any for document types
- Top-level PlaySession model at src/lib/server/models/PlaySession.ts creates indexes but appears unused while each game has its own PlaySession model

10.5 Nullable Issues

- User.email is default: null but some code treats it as always defined

- linkedProviders defaults to [] but code guards against undefined inconsistently

- pendingEmail and pendingPasswordHash are untyped (no | null constraint in schema definition)

- User.phone is default: null with no optional modifier on usage sites

- avatar is default: null used across multiple type-sensitive operations

10.6 Recommendations

1. Replace all `any` with specific types in auth helpers (formatUser, authPayload, issueSession, handleOAuth)

2. Create shared API response types to eliminate error format inconsistency

3. Add proper typing to all Mongoose model hooks instead of `as any` casts

4. Create UserDocument interface used consistently across all services

5. Consider generating TypeScript types from Zod schemas for automatic type safety

6. Remove PlaySession top-level model or refactor all games to use it as a base

7. Unify difficulty type definitions across all game-specific

files
\n### 11. Code Quality Audit

11.1 Dead Code & Unused Code
- The top-level PlaySession model (src/lib/server/models/PlaySession.ts) creates indexes (userId+puzzleId, userId+status, userId+status+completedAt) and a TTL on completedAt (90 days) but appears unused since each game has its own PlaySession model
- Legacy sudoku/crossmath/tangram routes (/sudoku/complete, /crossmath/complete, /tangram/complete) appear deprecated but still active alongside new patterns
- The src/data/ directory duplicates puzzle data that exists in shared/src/data/
- The top-level src/lib/server/models/PlaySession.ts model is a ghost collection — schema and indexes exist but no service code references it

11.2 Magic Values (Hardcoded)
- Password bcrypt salt rounds: 10 (hardcoded)
- JWT expiry times: 15m access, 7d refresh (configurable via env vars)
- Rate limit thresholds: 120 req/min for games/daily, 30 req/min for complete, 10 req/min for login, 3 req/min for register (not centrally configured)
- Max body size: 512KB default (configurable via MAX_BODY_SIZE env var)
- TTL for sessions: 7 days (hardcoded in LoginSession schema index)

- TTL for completed sessions: 90 days (hardcoded in PlaySession schema index)
- TTL for analytics: 180 days (hardcoded with env override ANALYTICS_RETENTION_DAYS)
- Brute force lockouts: 5 attempts / 5s lock; 10 / 30s lock; 20 / 5min lock (hardcoded)
- Rate limit windows: all 60,000ms (1 minute) — not configurable per endpoint

11.3 Long Methods (Top 3)
1. Auth route handler (~550 lines): 11 actions (register, login, logout, logout-all, refresh, change-password, set-username, link-and-merge, unlink-provider, manage-email, upgrade) in one function
2. handleOAuth() in authHelpers.ts (~150 lines): 5+ major branch paths (existing user, email merge, guest conversion, brand-new user, publicId backfill)
3. StatisticsService.updateUserStats() (~100 lines): per-difficulty stats tracking, streak calculation, favorite difficulty determination

11.4 Naming Inconsistency
- Session path parameter: [sessionId] (crossmath, tangram) vs [id] (nonogram, sudoku)
- Puzzle path parameter: puzzleId vs id
- Completion endpoint: complete vs completeSession vs completeRoute
- Rate limit function: checkRateLimit() vs rateLimit() (two wrappers doing similar things)

- Database model names: inconsistent — CrossMathPuzzle, NonogramPuzzle, TangramPuzzle with singular-named collections

11.5 Code Duplication (Most Costly Patterns)
1. Session route structure (complete/verify/save/pause/resume/restart/abandon/replay) — duplicated 4x across games, each file ~100-150 lines = ~440 lines of near-identical code
2. Rate limiting + auth + connectDB boilerplate at the top of every route handler — duplicated ~80+ times across all API routes
3. Error handling pattern (try/catch wrapping every handler) — duplicated across all route files
4. Statistics service update pattern — identical logic for updating user stats duplicated per game (CrossMathStatisticsService, NonogramStatisticsService, SudokuStatisticsService, TangramStatisticsService each repeat the same streak/completions/time tracking logic)
5. The withAuth wrapper pattern duplicated in crossmath/route-helpers.ts, nonogram/route-helpers.ts, tangram/route-helpers.ts, and the legacy route helpers

---

---

12. DevOps Audit

12.1 Docker & Containerization

- No Dockerfile found
- No docker-compose.yml found
- No docker-compose.yaml found
- No Kubernetes manifests (no k8s/ directory)
- Production deployment relies entirely on Vercel's serverless platform

12.2 CI/CD Pipeline
- No GitHub Actions workflow files (.github/workflows/ does not exist)
- No automated test execution on code changes
- No automated lint check
- No automated TypeScript type checking
- No automated build verification
- No automated security scanning (npm audit, Snyk, Dependabot)
- No deployment pipeline — manual Vercel deploy only

12.3 Monitoring & Observability
- No structured logging framework — console.log/console.error throughout codebase
- No distributed tracing — no OpenTelemetry, no correlation IDs
- No metrics collection — no Prometheus, no Datadog, no custom metrics
- No error aggregation — no Sentry, no LogRocket, no

error tracking service
- The auditLog() function is fire-and-forget with silent error swallowing (catch {})
- Analytics events use insertMany with ordered: false — events can be silently dropped with no retry
- No health check for external dependencies (Firebase, Stripe, Cloudinary)

12.4 Backup & Recovery
- No backup strategy documented
- No database backup automation
- No disaster recovery plan
- No backup verification or restore testing
- MongoDB Atlas may have automated backups but no application-level verification
- No point-in-time recovery capability verified

12.5 Secrets Management
- No secrets management tool (no HashiCorp Vault, AWS Secrets Manager, etc.)
- Secrets in .env.local for development and Vercel environment variables for production
- JWT secrets checked for minimum length (16 chars) but NOT for entropy/quality
- No JWT secret rotation strategy documented
- No secret versioning or rotation automation

- .env.local contains sensitive values and is in the repository (git-tracked) — potential secret exposure risk

12.6 Scaling

- No horizontal scaling configuration (relies on Vercel auto-scaling)
- No Kubernetes manifests (would be needed for self-hosted)
- In-memory rate limiting does NOT work across multiple instances
- In-memory brute force protection does NOT work across multiple instances
- In-memory completionBus does NOT work across multiple instances
- No session affinity configuration for stateful operations
- No connection pooling tuning beyond Mongoose defaults

---

13. Testing Audit

13.1 Test Files Found

1. src/data/sudoku/validate.test.ts — Sudoku dataset structural validation (9x9 grid, solution values 1-9, givens

match solution)
2. src/data/crossmath/validate.test.ts — CrossMath dataset structural validation (pattern existence, grid dimensions, blank cells, solution coverage)
3. src/data/sudoku/mockPuzzle.ts — mock puzzle data for tests
4. src/data/tangram/validate.test.ts — Tangram polygon dataset validation
5. src/test/nonogramDataset.test.ts — Nonogram dataset integrity (1000 puzzles per difficulty, clue-solution consistency, unique IDs, grid sizes)
6. src/test/toast-migration.test.ts — Toast migration logic test
7. src/test/setup.ts — Test setup (cleanup, jsdom matchMedia stub for components)
8. src/lib/server/puzzles/crossmath.test.ts — CrossMath puzzle solving logic test
9. src/lib/server/puzzles/daily.test.ts — Daily puzzle generation determinism test
10. src/lib/server/puzzles/serveSanity.test.ts — Serve-time sanity check tests
11. src/lib/server/seed/transform.test.ts — Seed data transformation test
12. src/lib/server/puzzles/sudoku.test.ts — likely exists for Sudoku solver testing
13. src/data/crossmath/index.test.ts — crossmath data

loading test
14. src/data/nonogram/validate.test.ts — likely exists for nonogram validation
15. src/components/games/sudoku/SudokuModal.test.tsx — client-side Sudoku modal test
16. src/lib/toast/useNetworkStatus.test.tsx — network status hook test
17. src/lib/toast/NetworkToastListener.test.tsx — network toast listener test
18. src/lib/toast/notify.test.ts — notification utility test

13.2 Testing Gaps — Critical
- No authentication flow tests (register, login, logout, refresh, OAuth callback)
- No API endpoint tests (no test files for any API route in src/app/api/)
- No authorization/access control tests (no tests for role-based permissions)
- No game session lifecycle tests (no tests for complete, save, verify, pause, resume)
- No security tests (no injection, XSS, CSRF, brute force, rate limiting tests)
- No CORS/origin validation tests
- No password reset/email verification flow tests
- No input validation edge case tests
- No error handling tests

- No concurrency/race condition tests
- No integration tests between service layers
- No E2E tests (Playwright test suite not configured)
- No performance/load tests
- No database migration tests
- No dataset integrity tests for production seed data

13.3 Coverage Estimate

Unit tests: ~15-20% of codebase (dataset structural tests + solver tests + integration tests)
Integration tests: ~0% (no integration test suite found)
E2E tests: ~0% (no Playwright test suite found)
Security tests: ~0%
Performance tests: ~0%

Testing Grade: D — The platform has structural validation tests for datasets and solver logic but lacks any meaningful API, auth, or integration test coverage.
\n### 14. Scalability Audit

14.1 User Scale Projections

User Count	Ready?	Bottleneck	Effort to Fix
10	Yes	None	N/A
100	Yes	Minor concerns	Minimal
1,000	Warning	In-memory state limits, connection pool	Moderate
10,000	Not ready	Memory + DB + rate limiting breaks	Significant

100,000+ | Not ready | Requires full infrastructure overhaul | Major
1M | Not ready | Would require complete redesign of state management | Very Major

14.2 Immediate Bottlenecks (100+ users)

1. In-memory rate limiting breaks on multi-instance deployments
2. In-memory brute force store breaks on multi-instance deployments
3. In-memory completionBus loses events on server restart, no multi-instance support
4. No database read replicas — all queries hit the single primary
5. O(n) streak calculation queries ALL completed sessions per user on every completion
6. Connection pool limit of 10 — insufficient for moderate concurrency
7. No CDN for puzzle data beyond Vercel edge network
8. No caching layer — every request hits MongoDB directly

14.3 Scale Barriers (10,000+ users)

1. MongoDB single instance — no replica set, no sharding, no read replicas documented
2. No Redis/caching layer — every puzzle request, session query, and leaderboard query hits the database
3. No CDN for static/puzzle data beyond Vercel's edge network
4. No horizontal scaling for API routes (Vercel auto-scales serverless functions, but MongoDB becomes the bottleneck)
5. Leaderboard pagination uses skip/limit which degrades severely at scale (skip 10000 scans 10000 rows)
6. Streak calculation runs full table scan per completion — O(n)

per user per play

14.4 Serverless Readiness (Vercel)
- Route handlers are stateless (aside from in-memory state) — good for serverless
- Database connections use global caching (mongoose connection caching) — reduces cold start impact
- Rate limiting is NOT serverless-ready (in-memory, per-instance, lost on scale-down)
- Brute force protection is NOT serverless-ready (same issue)
- Completion events are NOT serverless-ready (in-memory EventEmitter)
- Session fingerprinting works per-instance but not across instances

---

---

15. Bug Catalogue
ID | Severity | Category | Title | Evidence | Affected Files | Estimated Effort | Priority
----|----------|----------|-------|----------|----------------|-----------------|----------
BUG-001 | CRITICAL | API | Duplicate API Surface — legacy and modern endpoints coexist | Legacy routes exist at /sudoku/complete, /crossmath/complete, /tangram/complete alongside modern /games/[game]/sessions/[id]/* patterns | Multiple route files | 16h | High
BUG-002 | HIGH | Security | Weak password policy (6-char min, no complexity) | registerSchema min(6) in authValidator.ts | src/lib/server/validators/authValidator.ts | 8h | High

BUG-003 | HIGH | Security | No brute force protection on verification/password reset | bruteForce.ts only called in login flow, not in password reset or verification endpoints | src/app/api/v1/auth/[...slug]/route.ts | 8h | High
BUG-004 | HIGH | DevOps | No CI/CD pipeline | No .github/workflows/ directory exists | None (missing) | 12h | Critical
BUG-005 | HIGH | DevOps | No Dockerfile for containerized deployment | No Dockerfile or docker-compose found | None (missing) | 8h | Medium
BUG-006 | HIGH | Testing | No API test suite | Zero test files in src/app/api/ | None (missing) | 24h | Critical
BUG-007 | HIGH | Testing | No E2E test suite | No Playwright test files found | None (missing) | 32h | Critical
BUG-008 | HIGH | Performance | No Redis/caching layer | All DB queries direct, rate limiting in-memory | Multiple service files | 40h | High
BUG-009 | MEDIUM | Performance | Leaderboard uses skip/limit pagination | leaderboardQuerySchema uses numeric cursor (offset) | src/app/api/v1/games/[game]/leaderboard/route.ts | 12h | Medium
BUG-010 | MEDIUM | Security | CSP unsafe-eval/unsafe-inline weakens XSS protection | next.config.js CSP script-src includes 'unsafe-eval' and 'unsafe-inline' | next.config.js | 12h | Medium
BUG-011 | MEDIUM | Architecture | Auth route handler is god-class (~550 lines, 11 actions) | Single file handles register/login/logout/refresh/change-password/etc. | src/app/api/v1/auth/[...slug]/route.ts | 16h | Medium
BUG-012 | MEDIUM | Database | Duplicate PlaySession models | 4 game-specific+1 generic model; top-level model appears unused | src/lib/server/models/PlaySession.ts | 24h | Medium
BUG-013 | MEDIUM | Architecture | Dual puzzle data paths | src/data/ and shared/src/data/ contain overlapping puzzle data | src/data/*,

shared/src/data/* | 8h | Medium
BUG-014 | MEDIUM | Security | CSRF protection incomplete (cookie never set by server) | CSRF middleware checks header/cookie but never sets the cookie | src/lib/server/middleware/csrf.ts | 8h | Medium
BUG-015 | LOW | Code Quality | Inconsistent [id] vs [sessionId] naming | Nonogram uses [id], crossmath/tangram use [sessionId], sudoku uses [id] | Multiple session route directories | 4h | Low
BUG-016 | LOW | Code Quality | Unused top-level PlaySession model | Creates indexes and TTL but never referenced by any service | src/lib/server/models/PlaySession.ts | 4h | Low
BUG-017 | MEDIUM | Performance | O(n) streak calculation queries full session history | StatisticsService.updateUserStats queries ALL completed sessions per user on every completion | src/lib/server/puzzles/*/services/StatisticsService.ts | 8h | Medium
BUG-018 | LOW | TypeScript | Excessive any usage in auth helpers | formatUser, authPayload, issueSession, handleOAuth all use user: any | src/lib/server/utils/authHelpers.ts, formatUser.ts | 8h | Low
BUG-019 | MEDIUM | DevSecOps | No dependency vulnerability scanning | No Dependabot, Snyk, or npm audit in CI | None (missing) | 4h | High
BUG-020 | MEDIUM | Architecture | No API versioning strategy | /v1/ prefix but no deprecation, sunset dates, or version negotiation | N/A | 16h | Medium

---

---

16. Risk Matrix
Risk | Likelihood | Impact | Risk Score | Priority | Mitigation | Residual Risk
------|-----------|--------|-----------|----------|-----------|---------------

No CI/CD pipeline | High | Critical | 25 | P0 | Create GitHub Actions workflow | Low once implemented
No automated test coverage | High | High | 20 | P0 | Add Vitest + Playwright tests | Medium
Weak password policy | High | High | 20 | P0 | Enforce 8+ chars with complexity | Medium
No dependency vulnerability scanning | High | Critical | 20 | P0 | Dependabot + npm audit in CI | Medium
No backup strategy | Medium | Critical | 15 | P0 | MongoDB Atlas backups + verification | Medium
In-memory state breaks on scale | Medium | High | 15 | P1 | Implement Redis for all mutable state | Medium
Incomplete CSRF protection | Medium | High | 15 | P1 | Set CSRF cookie server-side | Low after fix
Missing brute force on auth endpoints | Medium | High | 15 | P1 | Add rate limiting to verification/reset | Low after fix
No monitoring/observability | High | High | 15 | P0 | Add Sentry + structured logging | Medium
CSP weakens XSS protection | Medium | Medium | 12 | P1 | Implement nonce-based CSP | Low after fix
Duplicate API surface | Medium | Medium | 12 | P1 | Consolidate legacy endpoints | Low after removal
Auth god-class hard to test/maintain | Medium | Medium | 10 | P2 | Decompose auth route | Low after refactoring
No Docker/containerization | Medium | Medium | 10 | P2 | Create Dockerfile | Low after creation
No API versioning governance | Medium | Medium | 10 | P2 | Establish versioning policy | Low after governance
CSRF fallback allows spoofed headers | Medium | Medium | 10 | P1 | Remove header spoof fallback | Low after fix
O(n) streak calculation | Low | Medium | 8 | P3 | Cache streaks

incrementally | Low after fix
Duplicate puzzle data paths | Medium | Low | 8 | P3 | Remove src/data/duplicates | Low after cleanup
No database migration framework | Medium | Medium | 8 | P2 | Add migration tooling | Low after setup
Avatar upload without validation | Medium | Medium | 10 | P2 | Add file type/size validation | Low after fix
JWT secret entropy unchecked | Low | Critical | 12 | P1 | Enforce 32+ char secrets with entropy check | Low after enforcement
\n---

---

17. Remediation Roadmap

Phase 0: Critical Blockers (1-2 weeks)

ID | Task | Owner | Effort | Business Impact
----|------|-------|--------|-----------------
BUG-004 | Create CI/CD pipeline (lint, typecheck, test, build) | DevOps | 12h | Prevents broken deployments
BUG-006 | Add API integration tests for auth + game endpoints | Backend | 24h | Catches regressions
BUG-007 | Add Playwright E2E tests for critical user journeys | QA | 32h | Prevents broken user flows
BUG-019 | Add Dependabot + npm audit to CI | DevOps | 4h | Catches vulnerable deps
****Phase 0 Total** | | | **72h** |**

Phase 1: Security Hardening (2-3 weeks)

ID | Task | Owner | Effort | Business Impact

----|------|-------|--------|----------------

BUG-002 | Strengthen password policy (8+ chars, complexity) | Backend | 8h | Reduces credential compromise

BUG-003 | Add brute force protection to verification/password reset | Backend | 8h | Prevents enumeration attacks

BUG-010 | Replace unsafe-eval/unsafe-inline CSP with nonces | Frontend | 12h | Strengthens XSS protection

BUG-014 | Fix CSRF cookie not set by server middleware | Backend | 8h | Prevents CSRF attacks

BUG-005 | Create Dockerfile for containerized deployment | DevOps | 8h | Enables portable deployments

BUG-011 | Decompose auth god-class into separate route files | Backend | 16h | Improves maintainability/testability

****Phase 1 Total** | | | **60h** |**

Phase 2: Performance & Scalability (3-4 weeks)

ID | Task | Owner | Effort | Business Impact

----|------|-------|--------|----------------

BUG-008 | Implement Redis caching layer for queries, rate limiting | Backend | 40h | Enables multi-instance scaling

BUG-001 | Consolidate/remove duplicate legacy API endpoints | Backend | 16h | Reduces maintenance burden

BUG-013 | Remove duplicate src/data/ puzzle files | Backend | 8h | Reduces maintenance risk

BUG-012 | Consolidate PlaySession models into single model | Backend | 24h | Reduces schema duplication

BUG-009 | Implement cursor-based pagination for leaderboards | Backend | 12h | Improves leaderboard perf at scale

BUG-017 | Cache streak calculations incrementally | Backend | 8h |

Improves completion perf
****Phase 2 Total** | | | **108h** |**

Phase 3: Observability & Operations (2-3 weeks)

ID | Task | Owner | Effort | Business Impact
----|------|-------|--------|-----------------
(all) | Add error tracking (Sentry/LogRocket) | DevOps | 12h | Enables production debugging
(all) | Add structured logging framework | DevOps | 16h | Enables production debugging
(all) | Implement backup verification automation | DevOps | 12h | Ensures data recoverability
(all) | Add health checks for external dependencies | DevOps | 8h | Enables proactive ops
(all) | Add database migration tooling | Backend | 16h | Enables schema evolution
BUG-020 | Implement API versioning governance | Backend | 16h | Enables controlled API evolution
****Phase 3 Total** | | | **80h** |**

Phase 4: Long-term Improvements (1-2 months)

ID | Task | Owner | Effort
----|------|-------|--------
All | Comprehensive game-specific test suites | QA | 40h
All | Load testing (Artillery/k6) for scalability | QA | 16h
All | Security scanning (Snyk/OWASP ZAP) in CI/CD | DevOps | 12h
— | Comprehensive API documentation | Backend | 16h
— | Create incident response runbooks | Ops | 12h

Phase 4 Total | | | **96h**

Total Estimated Effort: 316 engineering hours (~8-10 weeks for a 2-person team)

---

18. Final Verdict

Overall Production Readiness Score: 5.8/10 — Grade: C+

Launch Recommendation: CONDITIONAL — Limited Beta Only

The platform can be launched for a controlled beta with 10-100 users on Vercel while the critical Phase 0 and Phase 1 improvements are implemented. General availability should be deferred until CI/CD (Phase 0), security hardening (Phase 1), and monitoring (Phase 3) are complete.

Engineering Strengths (Top 10)

1. **Strong authentication design** — JWT lifecycle with refresh token rotation, session-based revocation, brute force protection, OAuth account linking
2. **Server-authoritative game verification** — all puzzle completion verified server-side, preventing client manipulation across all 4 games

3. **Zod validation at API boundaries — strong input contracts for all endpoints**
4. **Comprehensive production security headers — HSTS, XFO, X-Content-Type, CSP, Referrer-Policy, Permissions-Policy**
5. **Consistent response envelope — all APIs return {success, version, payload, serverTimestamp}**
6. **Audit logging — auth events and security-relevant actions are logged (fire-and-forget)**
7. **Rate limiting — all auth endpoints and game completion endpoints have rate limiting**
8. **Cookie security — httpOnly, Secure, SameSite=Strict on refresh tokens**
9. **Modern tech stack — Next.js 16, React 19, TypeScript 5 strict mode, Zod 4, TanStack Query**
10. **Sophisticated Tangram geometry engine — polygon geometry verification with rotation, flipping, snapping, containment, overlap detection — the strongest verification implementation of any game**

Engineering Weaknesses (Top 10)

1. No CI/CD pipeline — zero automated quality gates
2. No test suite — no API tests, E2E tests, or integration tests
3. No caching infrastructure — all DB queries direct, no Redis layer
4. In-memory state — rate limiting, brute force, completion events all use in-memory storage that breaks on multi-instance
5. No monitoring/observability — no error tracking, structured logging, or metrics
6. No backup strategy — no database backup automation or verification
7. No Docker — no reproducible deployment artifacts
8. No security scanning — no dependency vulnerability scanning in CI

9. Duplicate API surface — legacy and modern endpoints coexist creating maintenance hazard
10. God-class auth handler — 550-line auth route combining 11 actions in one function

Overall Production Readiness Score: 5.8/10

\n
